BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II: SESSION 2025/2026

Project Title:

MICROSERVICES-BASED DISASTER RELIEF COORDINATION PLATFORM

GROUP 16

COURSE LECTURER:

DR. MOHAMAD NOR HASSAN

NAME	MATRIC NUMBER
ADLY AZAMIN BIN AZMAN	S76094
MUHAMMAD UQAIL RAFIQIE BIN MOHD FARIS	S76227
AFIQ FAHIM BIN MOHD RIDZUAN	S76096

Table of Contents

Introduction

A microservice design was chosen because it makes the software flexible and strong. Instead of building one giant application where everything is tangled together, the system is broken down into small independent pieces. Each piece focuses on doing just one specific job. This means that a single part can be changed, tested and launched on its own without breaking the rest of the software. If one piece stops working, the other parts keep running smoothly so users are not interrupted.

This setup also changes how information is stored. Instead of forcing the whole system to use one massive database that can easily get slow, each small piece gets its own separate database. This lets developers pick the best tools for each specific job because these pieces run separately in the cloud making it very easy to handle heavy traffic. If lots of people suddenly use one specific feature, more computer power can be added just to that one part. This keeps the software running fast and saves money because there is no need to pay for an upgrade to the entire system at once.

Problem Description

During emergencies like floods, earthquakes or disease outbreaks, it is very important for different groups to work well together. However, this is often hard to do. Many current disaster systems use older single block designs. These older designs are not flexible enough to handle the sudden rush of users and information that happens during a crisis.

A big problem with these older systems is that they can slow down or crash completely when too many people use them at once. If many victims try to report problems or ask for help at the exact same time, the system can freeze. This can cause dangerous delays for rescue teams trying to save lives.

Another issue is poor communication between groups like the government, charities, and volunteers. When information is not shared instantly, rescue workers might end up doing the same job twice or they might completely miss areas that badly need help.

Also, updating these traditional systems is very hard because all the parts are tightly linked together. Changing just one small piece can break the whole system. This makes fixing or improving the software slow and risky during an emergency.

Because of these problems, a better, more flexible system is needed. A modern design must be able to handle sudden spikes in users, share information instantly and allow different parts of the system to work and update on their own without breaking the rest of the platform.

System Overview

The new Disaster Relief Platform is a modern system built to replace older, single-block apps used in crisis management. In the past, older disaster systems had all their software parts tightly linked together, meaning a change to one small piece could break the whole system. This made updates very slow and risky during a real emergency. A major problem with these older designs is that they often slow down or crash completely when too many victims try to report problems at the exact same time.

To fix these problems, the new platform uses a microservices design that separates the main jobs into completely independent pieces. There are separate parts for managing user accounts, handling emergency reports, tracking logistics, and coordinating volunteers. Because each part focuses on doing only one specific job, a single piece can be updated or fixed on its own without breaking the rest of the software. To connect everything, a central API Gateway acts as the main front door for the app, securely taking requests from users and instantly routing them to the correct background service.

Furthermore, instead of forcing the whole system to share one massive database that can easily get slow, each small piece is given its own separate database. This setup stops data traffic jams and is incredibly useful during unpredictable disasters. If a flood happens and thousands of people send emergency reports at once, the system can automatically add more computer power just to the reporting piece without needing to upgrade the entire platform. This targeted scaling keeps the rescue tools working fast, shares information instantly, and prevents dangerous delays to save lives.

System Objectives

I. Real-time processing

The system must ensure that critical data, such as emergency reports and location updates, is shared instantly among government bodies, charities, and volunteers. This allows the system to process information immediately so rescue teams can act fast without having to wait.

II. Scalability

The system should be able to handle sudden spikes in users and information, especially during a large-scale crisis. This is achieved by making sure each small, independent part of the software can grow on its own when it needs to handle heavy traffic.

III. Loose Coupling

The software should be designed so that each part can work by itself and is not too dependent on other parts. This makes it easier to update or fix one piece of the system, such as supply tracking, without breaking the rest of the platform.

IV. Automation of coordination tasks

The platform should automate tasks like tracking the distribution of food and medicine, and assigning volunteers to specific rescue jobs. This reduces mistakes, makes the rescue process more efficient, and allows rescue workers to focus on the important job of saving lives.

V. Improved user experience

The system must be reliable and easy to use for victims, volunteers, and agencies. Keeping users connected and ensuring the platform stays online during an emergency makes the whole disaster relief process smoother, more professional, and much more effective.

System Requirement

Functional Requirements

I. User Access & Security

The system shall allow victims, volunteers, and agencies to create and manage their own accounts and shall use a single entry point (API Gateway) to keep all services secure.

II. Emergency Reporting

The system shall allow users to send real-time reports with their location and the type of emergency and shall allow the Reporting Service to scale independently to handle traffic spikes

III. Logistics & Supplies

The system shall provide a specific service to track and manage the distribution of food and medicine and use a separate database for supply tracking to keep the data organized and fast.

IV. Volunteer Coordination

The system shall allow volunteers to sign up and be assigned to specific rescue tasks and shall allow the volunteer service to be updated without stopping the rest of the platform.

V. Communication

The system shall share information instantly between the government, charities, and volunteers to avoid missing areas in need and shall allow different parts of the software to talk to each other to keep information up to date.

Non-Functional Requirements

I. Performance

The platform must be fast. When people send emergency reports or update their locations, there should be no waiting. API response times must be very quick so rescue operations are not delayed.

II. Reliability

The system must stay online almost 100% of the time. Because lives are at stake, the platform must be safe from totally breaking; if the volunteer part fails, the emergency reporting part must keep working normally.

III. Scalability

The system must handle huge crowds of users at once. During a sudden disaster, the emergency reporting part must be able to grow to handle thousands of requests without slowing down the rest of the app.

IV. Security

Personal details of victims and volunteers must be protected with encryption. The main front door (API Gateway) must check passwords carefully so only approved groups can see sensitive rescue data.

V. Maintainability

Programmers must be able to fix bugs or add new features to one part of the app without turning the whole system off.

Team Member Roles

Name	Role	Description
Adly Azamin bin Azman	Project Manager	Define project scope and timeline, allocate responsibilities to team members, track progress and ensure the project is delivered on schedule.
Muhammad Uqail Rafiqie bin Mohd Faris	Database Admin	Responsible for developing, managing, and maintaining the database system. Controls user permissions, protects data security and handles backup and recovery processes.
Afiq Fahim bin Mohd Ridzuan	System Architect	Determine how services are separated, plan communication between services, select technologies and frameworks, ensure scalability and stability.

Architecture Style Explanation

The system uses a Microservices Architecture. This means the app is built as a set of small, separate pieces that do not rely heavily on each other.

I. **Loose Coupling**

The pieces work independently. The Volunteer part does not need to know how the Logistics part is built. They only talk to each other using simple, agreed-upon rules.

II. **Independent Deployability**

Developers can update or fix a specific piece without having to restart the entire platform.

III. **Decentralized Data Management**

Instead of sharing one giant database, each piece manages its own database. This stops data traffic jams and lets developers use the best type of database for each specific job.

IV. **Fault Isolation**

If a bug causes the communication piece to crash, the problem is trapped. The Emergency Reporting and Logistics pieces will continue to work normally so critical rescue operations do not stop.

V. **Granular Scalability**

Computer power can be added exactly where it is needed. If thousands of people are reporting a flood, only the Reporting piece is made bigger, which saves money and resources.

Service or Component Identification

I. **API Gateway**

The main front door for all mobile and web users. It checks security and directs traffic to the correct internal service.

II. **User Management Service**

Handles sign-ups, logins, and controls what victims, volunteers, and agencies are allowed to do.

III. **Emergency Reporting Service**

Collects crisis reports, saving important data like the user's location, how bad the emergency is, and what type it is.

IV. **Logistics & Supply Service**

Tracks food, water, and medicine to see where they are currently located and where they need to go.

V. **Volunteer Coordination Service**

Keeps track of volunteers, their skills, and matches them to jobs where they are needed.

VI. **Communication Service**

Sends real-time alerts and messages between the government, charities, and rescue workers in the field.

Service Boundaries

In a microservices architecture, each service has its own responsibility and focuses on one specific task. This makes the system easier to manage because every service works independently from the others. If one service needs to be updated or repaired, the other services can continue running without being affected. In the Disaster Relief Coordination Platform, each service has a clear boundary to avoid overlapping responsibilities.

I. API Gateway

The API Gateway is the main entry point of the system. All requests from victims, volunteers, and agencies first go through the API Gateway before being sent to the correct service. It is responsible for checking user requests and directing them to the appropriate microservice. However, it does not store any disaster information or perform the business logic of the system.

II. User Management Service

The User Management Service manages all user-related information. It allows users to register, log in, and manage their profiles. It also handles different user roles such as victims, volunteers, and agencies. This service only manages user information and does not handle emergency reports, volunteer assignments, or relief resources.

III. Emergency Reporting Service

The Emergency Reporting Service is responsible for receiving and managing emergency reports submitted by users. It stores information such as the disaster location, disaster type, severity level, and report status. This service only focuses on emergency reporting and does not manage user accounts, volunteers, or logistics.

IV. Logistics & Supply Service

The Logistics & Supply Service manages relief resources such as food, water, medicine, and emergency equipment. It keeps track of available supplies and updates the distribution of resources to affected areas. This service does not manage user accounts, volunteer information, or emergency reports.

V. Volunteer Coordination Service

The Volunteer Coordination Service manages volunteer information and assigns volunteers to disaster response tasks. It keeps records of volunteer availability, skills, and assigned activities. The service only focuses on volunteer management and does not manage logistics, emergency reports, or user authentication.

VI. Communication Service

The Communication Service is responsible for sending notifications and important updates to victims, volunteers, and agencies. It delivers messages such as emergency alerts, volunteer assignments, and resource updates through SMS, email, or other notification methods. This service does not manage disaster reports or store logistics information.

Data Ownership

In a microservices architecture, each service manages its own database. This allows every service to operate independently without directly accessing another service's data. If information is needed from another service, communication is done through REST APIs instead of sharing databases.

Service	Database	Data Owned
API Gateway	None	Routes requests and handles authentication only. It does not store application data.
User Management Service	User Database	User accounts, login credentials, user profiles, and user roles.
Emergency and Reporting Service	Reporting Database	Emergency reports, disaster type, location, severity level, report status, and timestamps
Logistics & Supply Service	Logistic Database	Food, water, medicine, emergency supplies, inventory records, and distribution information
Volunteer Coordination Service	Volunteer Database	Volunteer profiles, skills, availability, assigned tasks, and task status.
Communication Service	Communication Database	Notifications, alerts, SMS logs, email records, and message delivery status.

Communication Design

The Disaster Relief Coordination Platform uses REST APIs to allow communication between the microservices. All requests from users first pass through the API Gateway, which acts as the main entry point of the system. The API Gateway checks each request and forwards it to the appropriate service.

The microservices do not communicate by accessing each other's databases. Instead, when information is required from another service, a REST API request is sent to the corresponding service. This approach maintains the independence of each microservice and reduces tight coupling between system components.

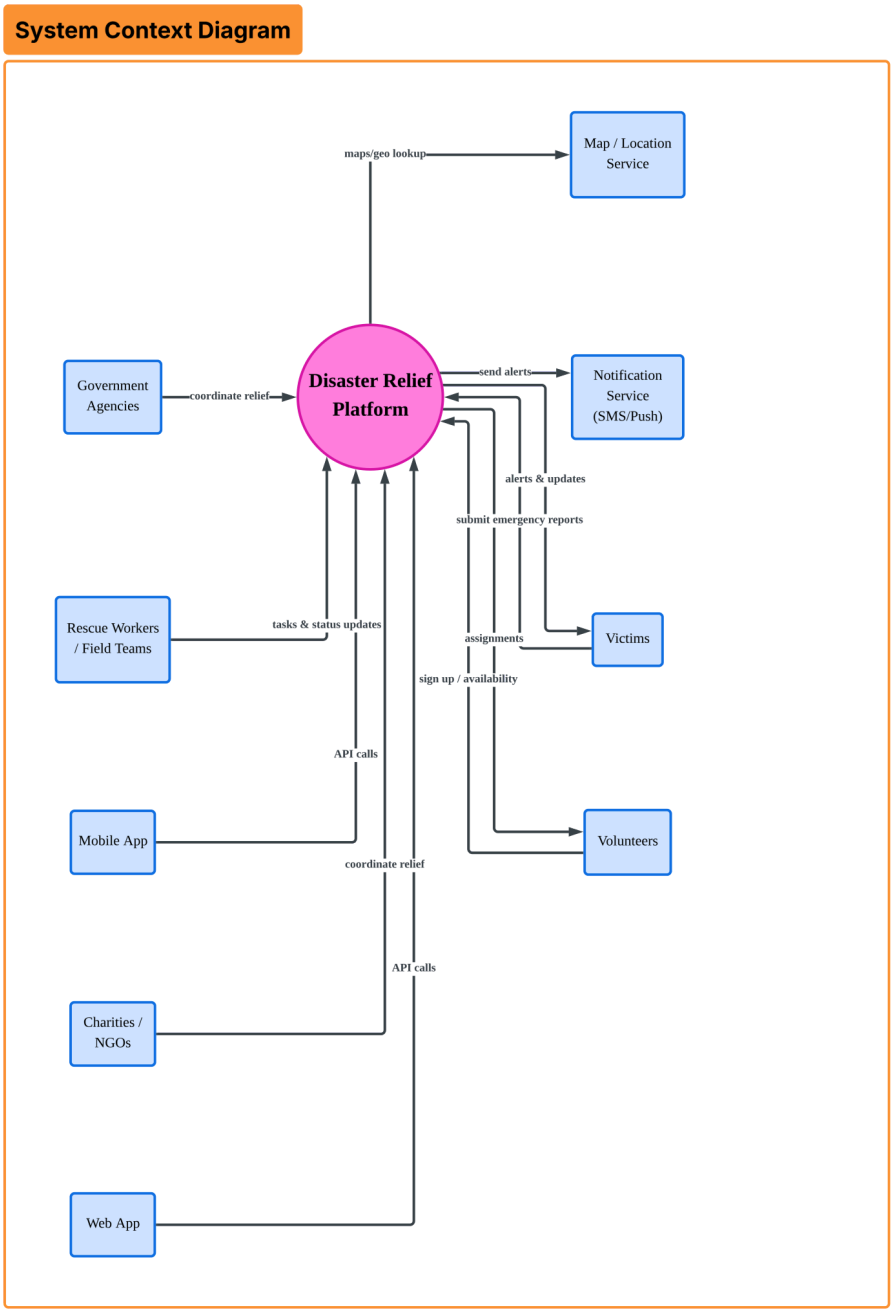
For example, when a victim submits an emergency report, the request is first sent to the API Gateway. The API Gateway forwards the request to the Emergency Reporting Service, where the report is stored. After the report is successfully recorded, the Emergency Reporting Service communicates with the Communication Service to send notifications and with the Volunteer Coordination Service to begin assigning available volunteers. If relief supplies are needed, the Logistics & Supply Service is also notified to prepare the required resources.

Using REST APIs ensures that each service can exchange information while remaining independent. This communication method improves system scalability, maintainability, and reliability because each service can continue operating even if another service is temporarily unavailable.

Source Service	Destination Service	Purpose	Communication Method
Client	API Gateway	Send user requests	HTTPS
API Gateway	User Management Service	User registration and login	REST API
API Gateway	Emergency Reporting Service	Submit emergency reports	REST API
Emergency Reporting Service	Communication Service	Send emergency reports	REST API
Emergency Reporting Service	Volunteer Coordination Service	Request volunteer assignment	REST API
Emergency Reporting Service	Logistic & Supply Service	Request relief resources	REST API

Architecture Diagrams

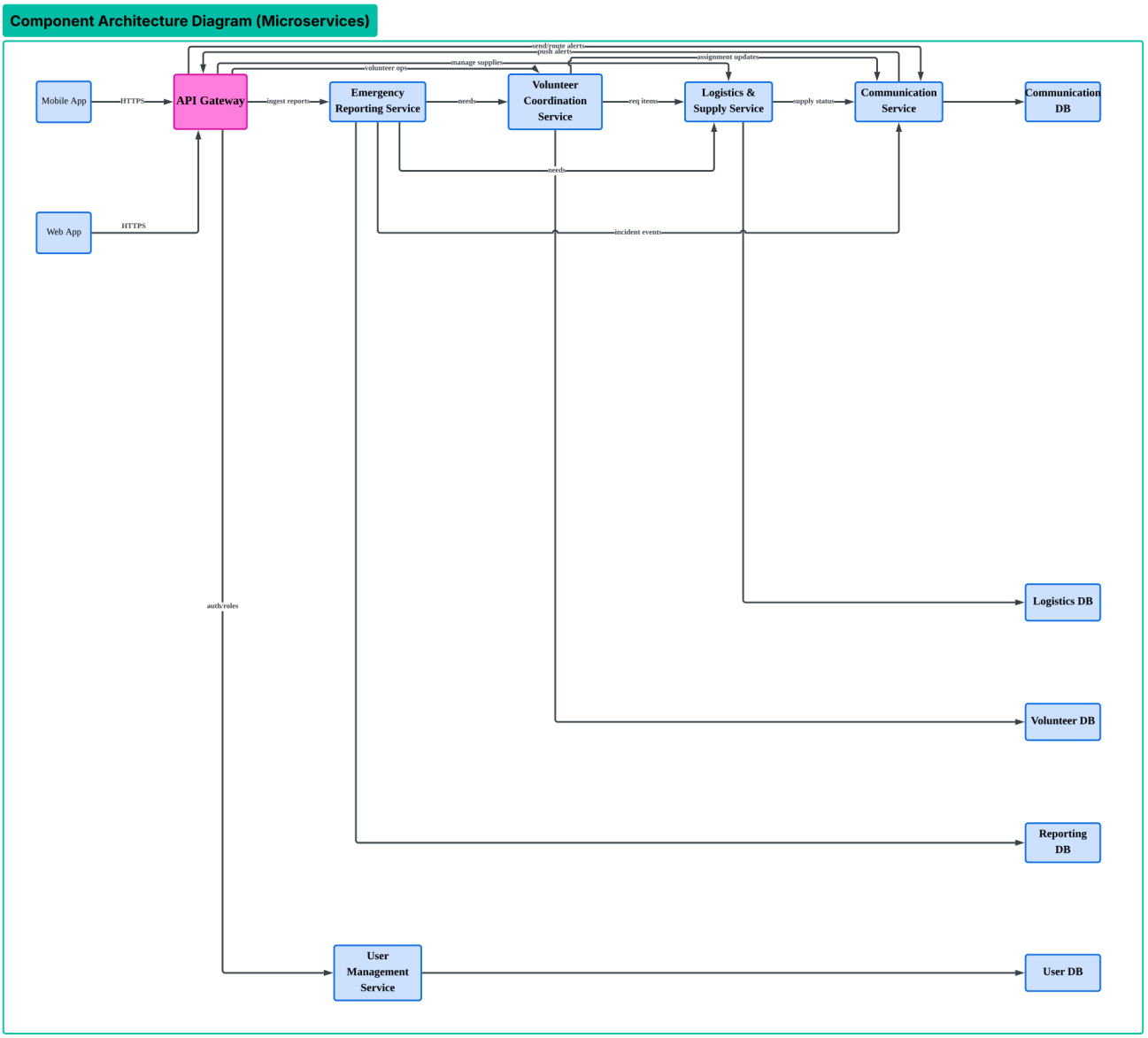
I. System Context Diagram



The above figure shows the high-level context of the Disaster Relief Platform, where victims, volunteers and agencies interact with the system to report emergencies, manage resources and coordinate tasks.

The platform also connects with external systems such as the Map System for location data and the SMS System for notifications. These interactions support real-time communication, fast response and efficient disaster management.

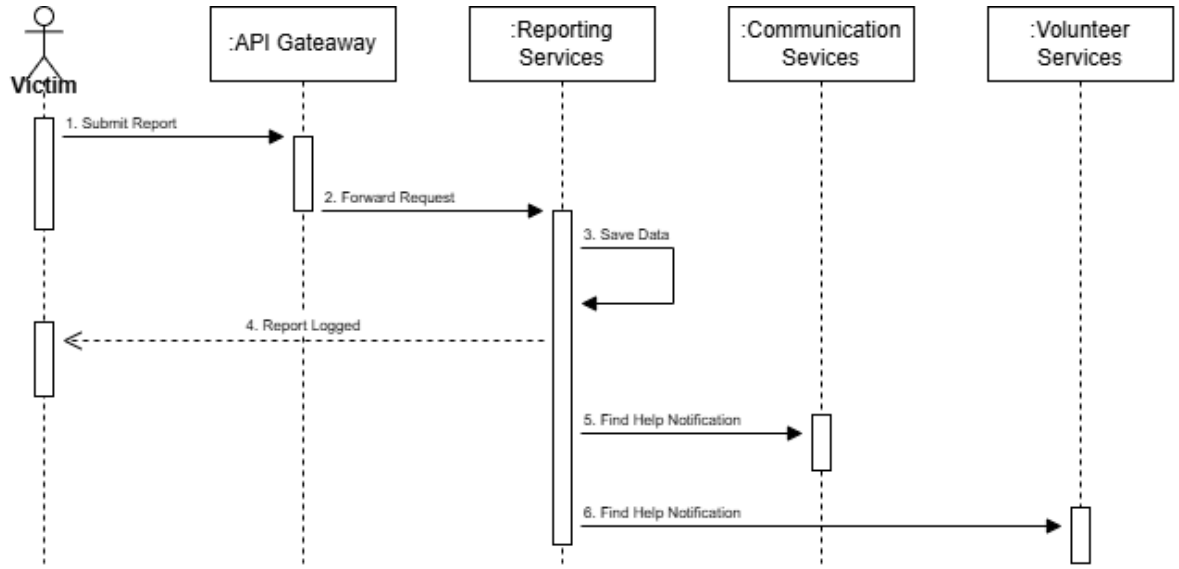
II. Component Architecture Diagram



The above figure illustrates the component architecture of the Disaster Relief Platform using a microservices design. The API Gateway acts as a central entry point that routes requests to services such as User, Reporting, Logistics, Volunteer and Communication services.

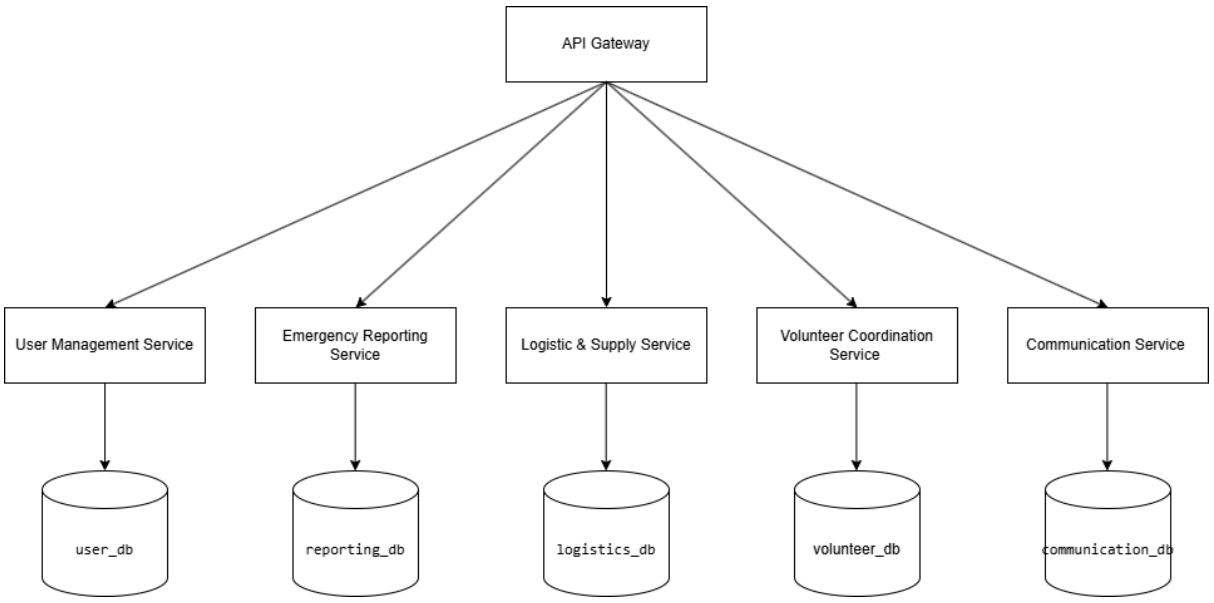
Each service has its own database, ensuring better scalability, reliability and independent operation of system components.

III. Interaction Diagram



The Interaction Diagram shows the step-by-step process of how the platform handles an emergency. It starts when a Victim sends a report through the API Gateway, which acts as the system's secure entry point. This gateway forwards the request to the Reporting Service, where the incident details are saved to a private database. After the data is stored, the system sends a "Report Logged" confirmation back to the Victim to let them know help is on the way. To finish the process, the Reporting Service automatically alerts the Communication and Volunteer Services to start organizing rescue efforts and finding available help instantly.

IV. Deployment Diagram



The Deployment Diagram shows how the platform is structured on a server to stay fast and reliable. At the top, the API Gateway acts as the main entry point, securely directing all user traffic to the right place. Below that, the system is split into five independent services, which allows the platform to keep running even if one part has a problem. To prevent data slowdowns, each service has its own dedicated private database, ensuring that information stays organized and secure during a major disaster.

Technology Stack

The Disaster Relief Coordination Platform was developed using Java-based web technologies. Each technology was selected based on its ability to support a microservices architecture while providing reliable performance, maintainability, and ease of development. The following table describes the technologies used in this project.

Technology	Purposes
Java	Main programming language used to develop the backend services.
Java Servlet	Handles HTTP requests, business logic, and communication between the client and the server.
JSP (Java Server Pages)	Creates dynamic web pages for users to interact with the system.
HTML	Builds the structure of the web pages.
CSS	Improves the appearance and layout of the user interface.
Java Script	Provides client-side validation and interactive features.
Apache Tomcat	Hosts and runs the web application.
MySql	Stores user information, emergency reports, volunteer data, and logistics records.
JDBC	Connects the Java application to the MySQL database.
Netbeans IDE	Used to develop and manage the project.

The technologies listed above work together to support the implementation of the Disaster Relief Coordination Platform. Java Servlets handle the business logic, JSP provides the user interface, Apache Tomcat hosts the application, and MySQL stores the system data. Together, these technologies provide a reliable environment for implementing the proposed microservices architecture.

Prototype Implementation

Login Module

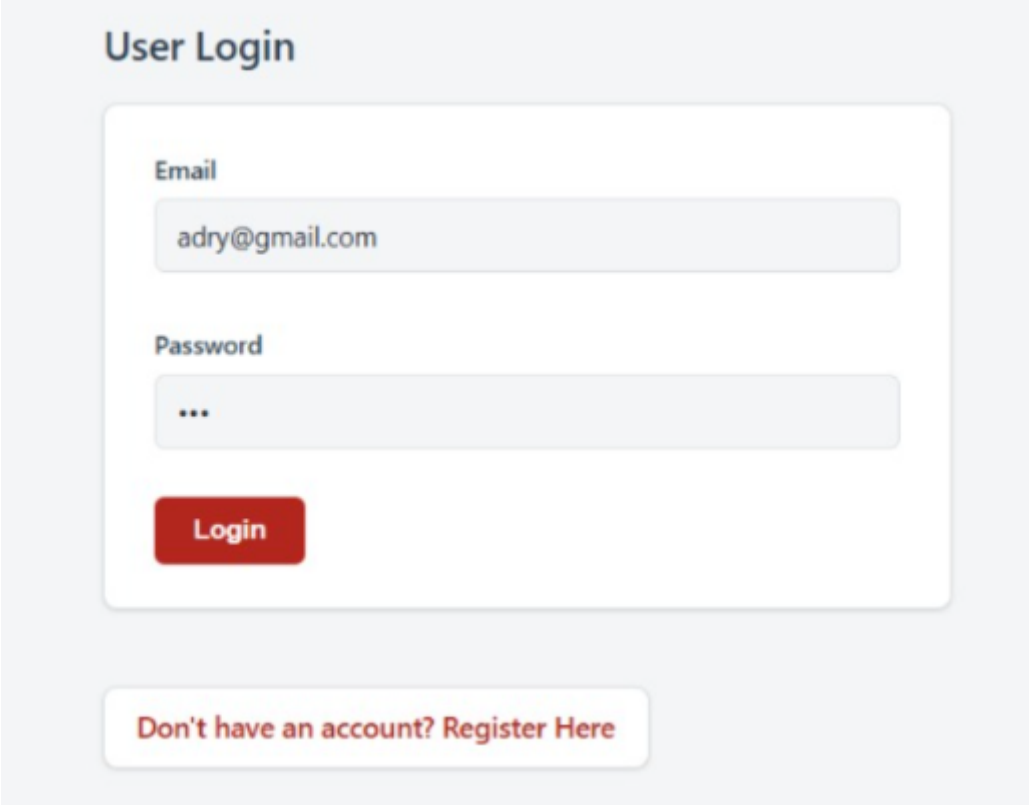


Figure 15.2 shows the login page of the Disaster Relief Coordination Platform. This page allows registered users to access the system by entering their email address and password. The login interface is designed to be simple and easy to use, allowing users to access the platform quickly during emergency situations.

When the user clicks the Login button, the system verifies the entered credentials against the user information stored in the database. If the login details are correct, the user is successfully authenticated and redirected to the Home page. If the credentials are incorrect, the system displays an error message and prompts the user to try again.

A Register Here button is also provided for users who do not have an account. This allows new users to create an account before accessing the services provided by the Disaster Relief Coordination Platform.

Registration Module

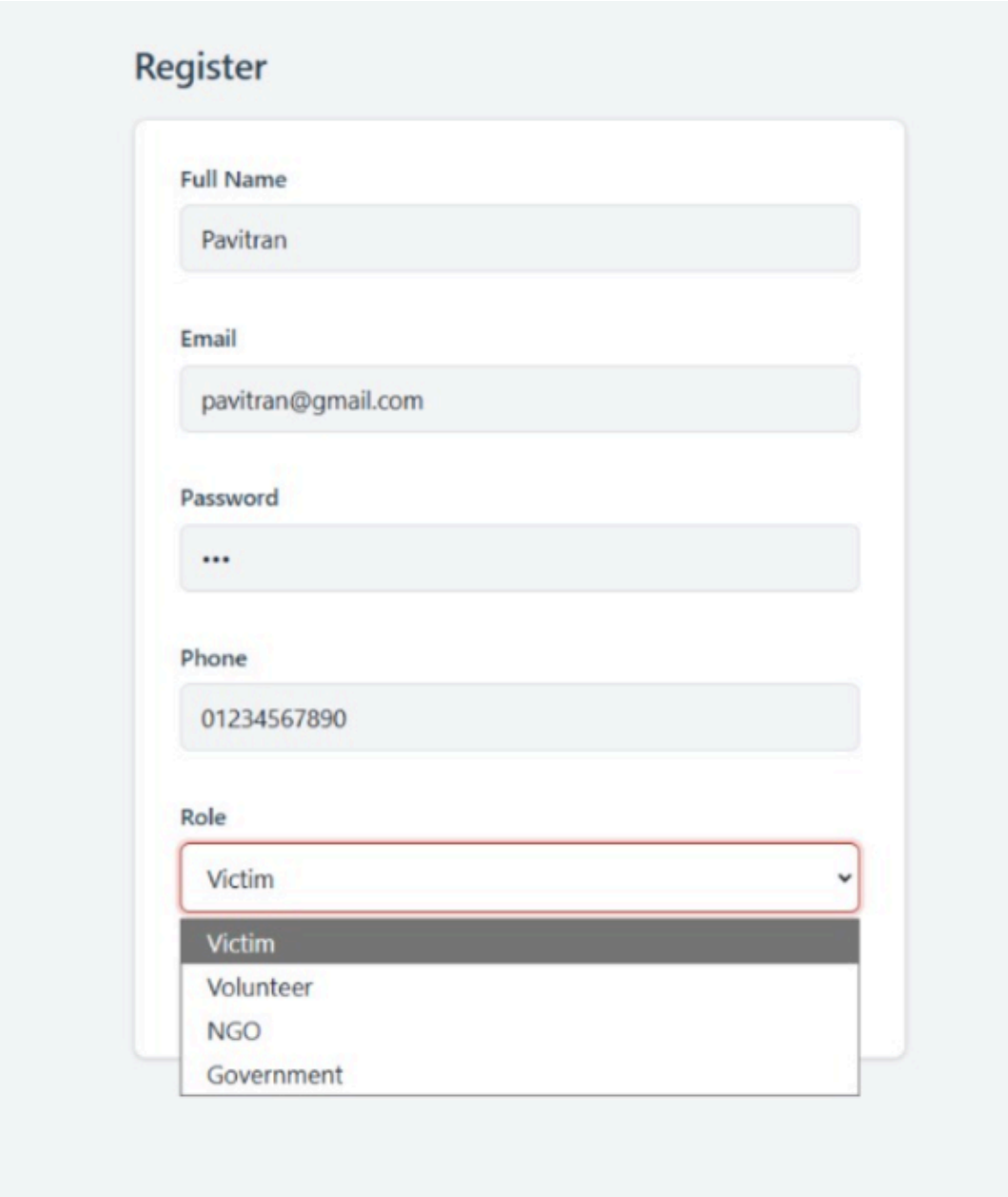


Figure 15.3 shows the user registration page of the Disaster Relief Coordination Platform. This page allows new users to create an account before accessing the system. During registration, users are required to enter their full name, email address, password, phone number, and select their role in the platform.

The system supports different user roles, including Victim, Volunteer, NGO, and Government. Each role is assigned different responsibilities and permissions within the system. For example, victims can submit emergency reports, volunteers can view and accept assigned tasks, while NGOs and government agencies can manage resources and coordinate disaster relief operations.

After the user submits the registration form, the system validates the entered information before storing it securely in the user database. Once the registration is completed successfully, the user can log in using the registered email address and password to access the Disaster Relief Coordination Platform.

Emergency Reporting

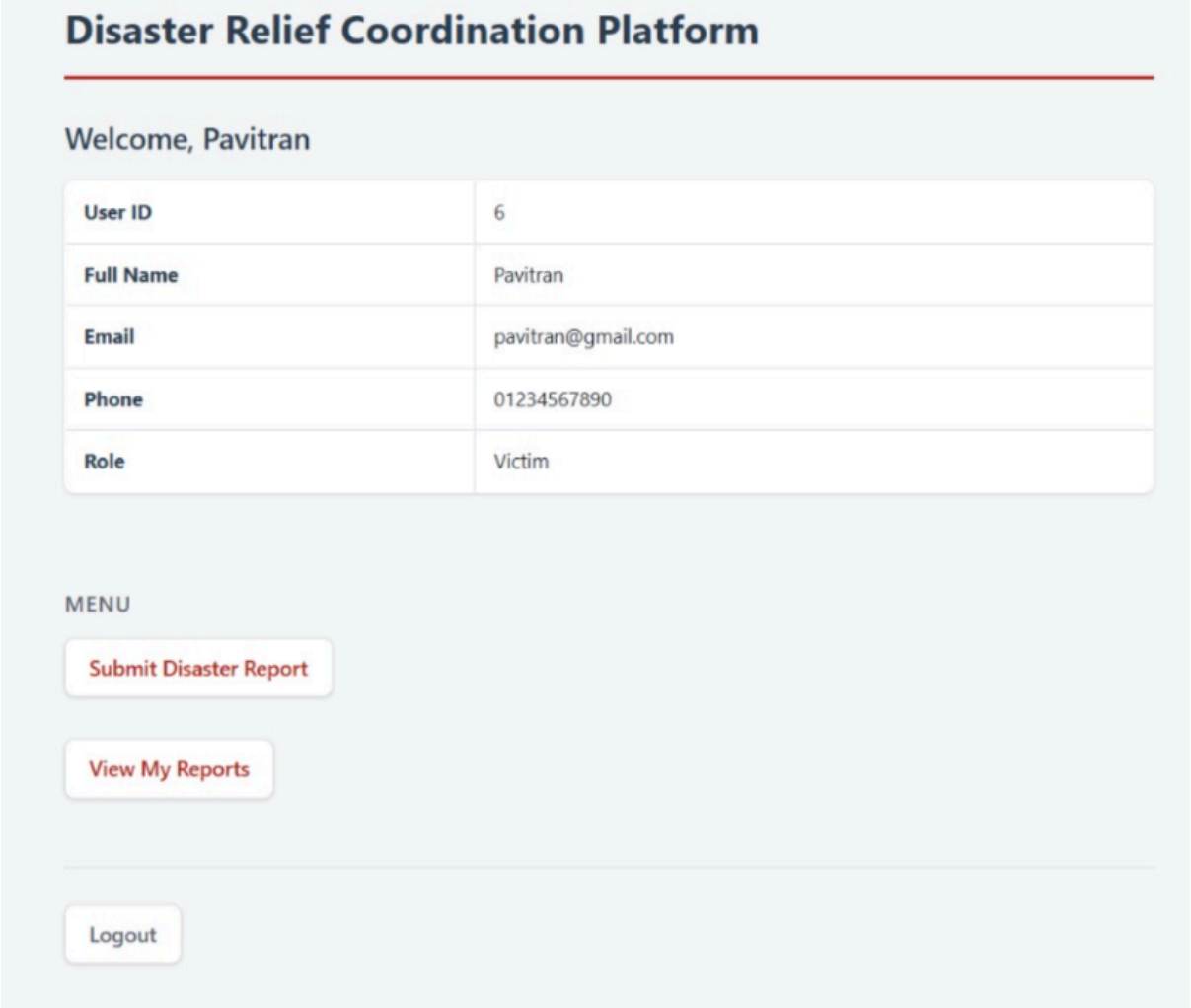


Figure 15.4 shows the Home Dashboard of the Disaster Relief Coordination Platform after a user successfully logs into the system. The dashboard serves as the main page where users can view their account information and access the available system functions.

The dashboard displays the user's profile details, including the User ID, Full Name, Email Address, Phone Number, and assigned Role. This allows users to verify that they have logged into the correct account. Based on the assigned role, users are provided with the appropriate features and permissions within the system.

The navigation menu provides quick access to the main functions of the platform. In this example, a victim can submit a new disaster report and view previously submitted reports. A Logout button is also provided to allow users to end their session securely after completing their activities.

Resource Management

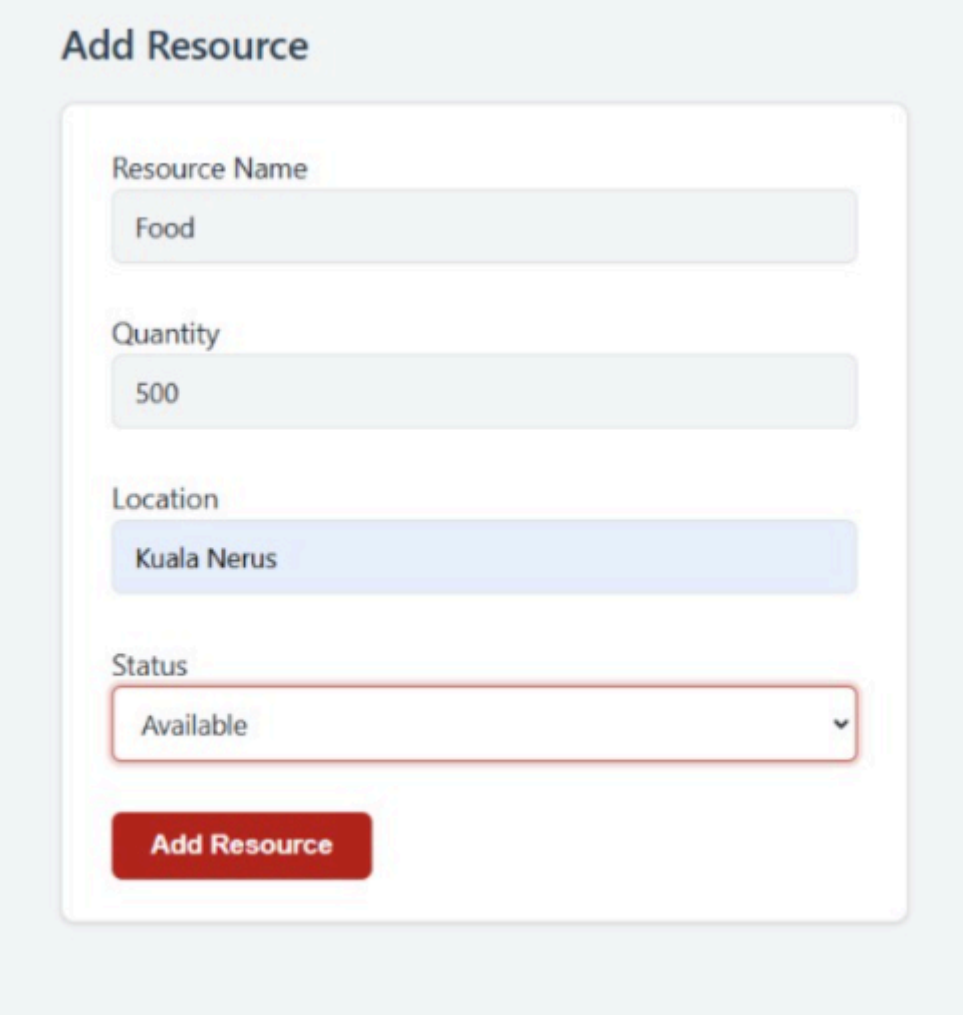


Figure 15.5 shows the Resource Management page of the Disaster Relief Coordination Platform. This module allows authorized users, such as government agencies and NGOs, to record and manage relief resources during disaster situations. The information entered includes the resource name, quantity, location, and current availability status.

After the user submits the form, the system validates the entered information before storing it in the Logistics database. This enables the platform to maintain an up-to-date record of available resources that can be distributed to affected areas. Keeping resource information updated helps improve coordination between relief agencies and ensures that supplies are allocated efficiently.

The Resource Management module supports the microservices architecture by allowing the Logistics & Supply Service to manage resource information independently without affecting other services such as Emergency Reporting or Volunteer Coordination.

Volunteer Tasks

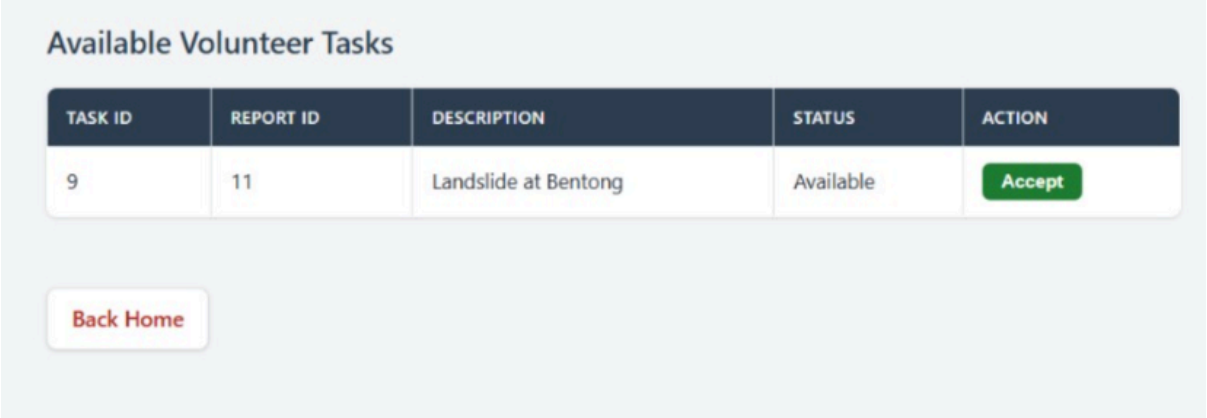


Figure 15.6 shows the Volunteer Task Management page of the Disaster Relief Coordination Platform. This module allows volunteers to view available disaster response tasks assigned by the system. Each task includes important information such as the Task ID, Report ID, task description, and its current status.

Volunteers can review the available tasks and click the Accept button to participate in a disaster relief operation. Once a task is accepted, the system updates the task status in the database to indicate that it has been assigned to a volunteer. This helps prevent multiple volunteers from accepting the same task and improves the coordination of rescue activities.

The Volunteer Task Management module supports the microservices architecture by allowing the Volunteer Coordination Service to manage volunteer assignments independently. This enables volunteer information and task assignments to be updated without affecting other services within the Disaster Relief Coordination Platform.

Notifications

Figure 15.7 shows the Notification Module of the Disaster Relief Coordination Platform. This module is responsible for displaying important notifications and emergency alerts to users. The notification list includes information such as the notification ID, user ID, message, notification type, and delivery status.

Whenever an emergency report is submitted or an important event occurs, the Communication Service generates a notification and records it in the database. These notifications help keep victims, volunteers, government agencies, and NGOs informed about ongoing disaster situations and response activities. The notification status also indicates whether the message has been successfully delivered.

The Notification Module supports real-time communication between different users of the platform and improves coordination during disaster response. By separating notification management into its own service, the system follows the principles of microservices architecture, allowing the Communication Service to operate independently without affecting other system components.

Architecture Evaluation

Scalability

The Disaster Relief Coordination Platform was designed using a microservices architecture to support scalability. Each service is responsible for a specific function, allowing individual services to be scaled independently according to system demand. For example, during a disaster where many victims submit emergency reports simultaneously, the Emergency Reporting Service can be expanded without affecting the User Management or Volunteer Coordination services. This approach improves system performance and ensures that critical services remain responsive during periods of high traffic.

Reliability

Reliability is an important requirement for disaster management systems because users depend on the platform during emergency situations. By separating the system into independent services, failures in one service are less likely to affect the operation of the entire platform. For example, if the Notification Service experiences temporary issues, users can still register, submit reports, and manage resources. This improves the overall availability of the system.

Maintainability

The platform is easier to maintain because each service has a clearly defined responsibility. Changes made to one service can be implemented without significantly affecting the others. The project structure, which separates controllers, models, services, and database access classes, also improves code readability and simplifies future development. This allows new features to be added more efficiently while reducing the risk of introducing errors into other parts of the system.

Security

Security is provided through user authentication and role-based access. Only registered users are allowed to access the system, and different user roles such as Victim, Volunteer, NGO, and Government have different permissions. The API Gateway acts as the main entry point to the system, helping control access to the available services. This reduces unauthorized access and protects sensitive disaster information.

Overall Evaluation

Overall, the Disaster Relief Coordination Platform successfully demonstrates the implementation of a microservices architecture. The system separates major functions into independent services, each responsible for a specific business function. This architecture improves scalability, reliability, maintainability, and flexibility compared to a traditional monolithic system. Although the project is developed as a prototype, the architecture provides a strong foundation for future enhancements and deployment in real-world disaster management scenarios.

Challenges Faced

During the development of the Disaster Relief Coordination Platform, the team encountered several challenges related to designing and implementing a microservices-based system. One of the main challenges was understanding how different services should communicate while remaining independent. Since each service is responsible for a specific function, careful planning was required to ensure that information could be shared efficiently without creating unnecessary dependencies.

Another challenge was designing separate databases for different services. It was important to decide which data belonged to each service while avoiding duplication and maintaining consistency across the platform. Proper database design was necessary to support communication between the services through REST APIs instead of direct database access.

The team also faced challenges during implementation, particularly in integrating the frontend with the backend. Developing the JSP pages, Java Servlets, and MySQL database together required careful testing to ensure that user requests were processed correctly and that data was stored accurately.

Time management was another challenge throughout the project. Developing multiple system modules, preparing architecture diagrams, and documenting the project within the given timeline required effective teamwork and coordination. Regular discussions among team members helped ensure that tasks were completed according to schedule.

Overall, these challenges provided valuable learning experiences and improved the team's understanding of software architecture, system integration, and collaborative software development.

Future Improvements

Although the Disaster Relief Coordination Platform successfully demonstrates the proposed microservices architecture, several improvements can be made in future development.

One possible improvement is the integration of a real-time mapping service to display the locations of victims, volunteers, and relief resources on an interactive map. This would allow emergency responders to identify affected areas more quickly and improve the coordination of rescue operations.

Another improvement is the implementation of push notifications and SMS services to deliver emergency alerts instantly to users. This would provide faster communication during disaster situations and ensure that important information reaches the appropriate users without delay.

The platform could also be enhanced by introducing role-based dashboards that provide different interfaces for victims, volunteers, NGOs, and government agencies. This would improve the user experience by displaying only the functions that are relevant to each user role.

Finally, the system could be deployed using containerization technologies such as Docker and Kubernetes. This would allow each microservice to be deployed, managed, and scaled independently in a cloud environment, making the platform more suitable for real-world disaster management.

Conclusion

The Disaster Relief Coordination Platform was developed to address the challenges of coordinating disaster response activities using a microservices architecture. By separating the system into independent services, the platform provides better scalability, reliability, maintainability, and flexibility compared to a traditional monolithic architecture.

The proposed system allows victims, volunteers, NGOs, and government agencies to collaborate more effectively through features such as user management, emergency reporting, resource management, volunteer coordination, and emergency notifications. Each service performs a specific responsibility while communicating through REST APIs, making the overall system easier to maintain and extend.

The implementation of the prototype demonstrates that the proposed architecture is capable of supporting disaster relief coordination in an organized and efficient manner. Although the project is developed as a prototype, it provides a strong foundation for future enhancements and practical deployment. Overall, this project has strengthened the team's understanding of software architecture principles, microservices design, and collaborative software development.

References

1. Apache Software Foundation. (n.d.). *Apache Tomcat documentation*. https://tomcat.apache.org/
2. Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
3. Bro Code. (2023). *Java full course for beginners* [Video]. YouTube. https://www.youtube.com/@BroCodez
4. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm
5. GeeksforGeeks. (n.d.). *Java programming language*. https://www.geeksforgeeks.org/java/
6. Java Guides. (2023). *JSP and Servlet tutorial* [Video]. YouTube. https://www.youtube.com/@JavaGuides
7. Newman, S. (2021). *Building microservices* (2nd ed.). O'Reilly Media.
8. Oracle. (n.d.). *Java documentation*. https://docs.oracle.com/en/java/
9. Oracle Corporation. (2024). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/
10. Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
11. Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.
12. Telusko. (2023). *Java full course* [Video]. YouTube. https://www.youtube.com/@Telusko